ClaimSight — Detecção de Danos em Imagens de Sinistros

Escopo Técnico Completo de Construção (Blueprint do Projeto)

> **Posicionamento honesto do projeto:** ClaimSight é uma ferramenta de **triagem** de sinistros automotivos com **revisão humana obrigatória**. O modelo realiza **classificação binária** (dano vs. íntegro) e indica a região que motivou a decisão via **Grad-CAM (localização fraca / explicabilidade visual)** — não é segmentação real. Segmentação verdadeira é uma extensão planejada.

---

1. Problema de Negócio e Objetivo Técnico

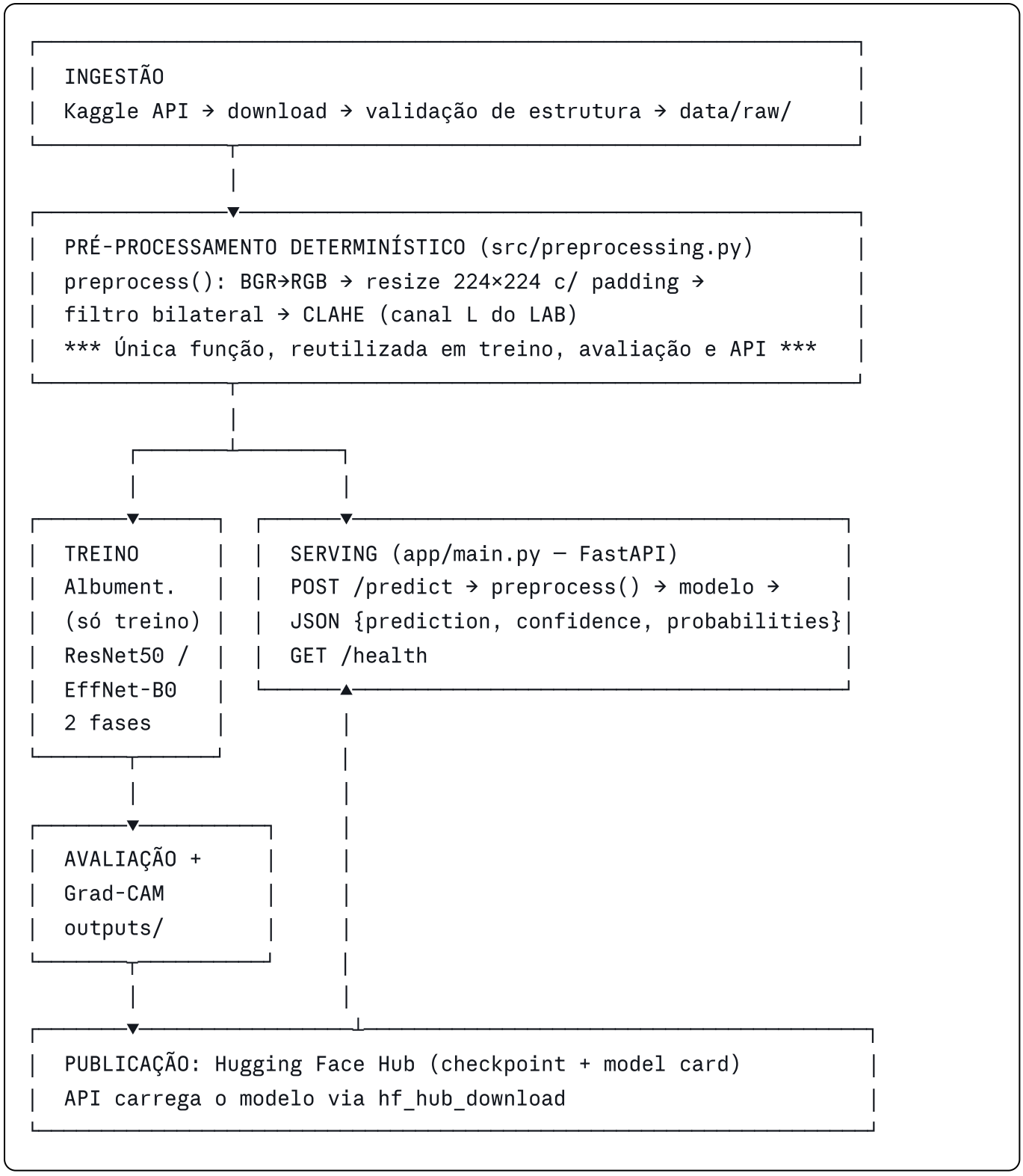
Seguradoras recebem grandes volumes de fotos de sinistros. A triagem manual inicial (há dano visível ou não?) consome tempo de analistas em casos triviais. O ClaimSight automatiza esse primeiro filtro: dado uma foto de veículo, o sistema retorna a probabilidade de dano, a classe predita e um mapa visual (Grad-CAM) indicando a região que mais influenciou a decisão.

O erro mais custoso do negócio é o **falso negativo** — um veículo danificado classificado como íntegro pode encerrar indevidamente um sinistro legítimo. Por isso, toda a avaliação é orientada a **recall da classe dano**, com ajuste explícito de threshold de decisão e documentação do trade-off precision/recall. A decisão final permanece sempre com o analista humano.

Objetivo técnico mensurável: classificador binário com val_accuracy alvo na faixa de 88–95% *(valor real: [MÉTRICA — preencher após treino])*, servido por API REST com pipeline de pré-processamento idêntico entre treino e inferência (zero training/serving skew).

---

2. Arquitetura da Solução (visão end-to-end)



O diferencial de engenharia está no ponto central do diagrama: `preprocess()` é a **única fonte de verdade** para transformar pixels em tensor. Treino, avaliação e API importam a mesma função do mesmo módulo, eliminando a categoria inteira de bugs de training/serving skew.

---

3. Estrutura do Repositório

```
claimsight/
├── data/
│   ├── raw/                  # download bruto do Kaggle (gitignored)
│   └── processed/            # opcional: cache de tensores
├── src/
│   ├── __init__.py
│   ├── ingestion.py          # download e validação via Kaggle API
│   ├── preprocessing.py      # preprocess() — fonte única de verdade
│   ├── dataset.py            # CarDamageDataset + transforms
│   ├── model.py              # build_model() para ResNet50/EffNet-B0
│   ├── train.py              # loop de treino em duas fases
│   ├── evaluate.py           # métricas, matriz de confusão, threshold
│   └── explain.py            # Grad-CAM overlays
├── app/
│   ├── main.py               # FastAPI (GET /health, POST /predict)
│   └── schemas.py            # Pydantic models de request/response
├── models/                   # checkpoints locais (gitignored)
├── outputs/                  # curvas, matrizes, overlays Grad-CAM
├── notebooks/                # exploração (EDA) — nunca lógica de produção
├── scripts/
│   └── publish_hf.py         # upload versionado para o HF Hub
├── tests/
│   ├── test_preprocessing.py # determinismo e shape do preprocess()
│   └── test_api.py           # contrato da API (TestClient)
├── requirements.txt          # versões pinadas
├── README.md
└── .gitignore
```

A separação `src/` (experimento) vs. `app/` (serving) é deliberada: o serving importa de `src/` apenas o que é contrato estável (`preprocess()`, `build_model()`), nunca código de treino.

---

4. Fase 1 — Ingestão de Dados (`src/ingestion.py`)

Dataset: Car Damage Detection (Kaggle, `anujms/car-damage-detection`). Estrutura: pastas `training/` e `validation/`, cada uma com subpastas `00-damage` e `01-whole`. **Apenas labels binários por pasta — não há máscaras nem bounding boxes**, o que fundamenta a decisão de arquitetura (classificação + Grad-CAM, não segmentação).

A ingestão é automatizada e idempotente: se os dados já existem e passam na validação, o download é pulado.

```
python
```

```python
import os
import zipfile
from pathlib import Path

DATASET_SLUG = "anujms/car-damage-detection"
RAW_DIR = Path("data/raw")
EXPECTED_DIRS = [
    "training/00-damage", "training/01-whole",
    "validation/00-damage", "validation/01-whole",
]

def download_dataset(force: bool = False) -> Path:
    """Download and extract the Kaggle dataset if not already present.

    Requires KAGGLE_USERNAME/KAGGLE_KEY env vars or ~/.kaggle/kaggle.json.
    Returns the root directory containing the extracted splits.
    """
    if not force and validate_structure(RAW_DIR):
        print("Dataset already present and valid — skipping download.")
        return RAW_DIR

    from kaggle.api.kaggle_api_extended import KaggleApi
    api = KaggleApi()
    api.authenticate()
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    api.dataset_download_files(DATASET_SLUG, path=str(RAW_DIR), unzip=True)
    assert validate_structure(RAW_DIR), "Downloaded dataset failed validation."
    return RAW_DIR

def validate_structure(root: Path) -> bool:
    """Check that all expected class folders exist and contain images."""
    for rel in EXPECTED_DIRS:
        folder = root / rel
        if not folder.is_dir():
            return False
        images = list(folder.glob("*.jpg")) + list(folder.glob("*.JPEG")) \
                 + list(folder.glob("*.png"))
        if len(images) == 0:
            return False
    return True

def dataset_summary(root: Path) -> dict:
    """Return image counts per split/class for the README and sanity checks."""
    return {
```

```
        rel: len(list((root / rel).iterdir())) for rel in EXPECTED_DIRS
    }
```

`dataset_summary()` alimenta o README com os tamanhos reais *([TAMANHOS — preencher com a saída real])* — nunca números inventados.

---

5. Fase 2 — Pré-processamento Determinístico (`src/preprocessing.py`)

Este é o coração do ângulo de Data Engineering. A função `preprocess()` é **determinística** (mesma entrada → mesma saída, sempre) e **compartilhada** entre treino, avaliação e API. Cada etapa tem justificativa técnica:

1. **BGR→RGB:** OpenCV lê em BGR; os pesos ImageNet do PyTorch esperam RGB. Esquecer essa conversão é um bug silencioso clássico que degrada acurácia sem lançar erro.
2. **Resize 224×224 com padding (letterbox):** redimensiona preservando a proporção da imagem e preenche as bordas, evitando distorcer a geometria do dano (um amassado esticado deixa de parecer um amassado).
3. **Filtro bilateral:** suaviza ruído de compressão JPEG preservando bordas — e bordas são exatamente onde danos aparecem (riscos, amassados, trincas).
4. **CLAHE no canal L (espaço LAB):** equaliza contraste local sem estourar cores. Fotos de sinistro vêm de celulares em condições de luz muito variáveis; CLAHE normaliza essa variabilidade.

```
python
```

```python
import cv2
import numpy as np

TARGET_SIZE = 224

def preprocess(image_bgr: np.ndarray) -> np.ndarray:
    """Deterministic preprocessing shared by training, evaluation and serving.

    Steps: BGR->RGB, aspect-preserving resize with padding to 224x224,
    bilateral filtering (edge-preserving denoise), CLAHE on the L channel
    of LAB color space (local contrast normalization).

    Args:
        image_bgr: raw image as loaded by cv2.imread (H, W, 3), dtype uint8.

    Returns:
        RGB uint8 array of shape (224, 224, 3).
    """
    image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
    image_rgb = _resize_with_padding(image_rgb, TARGET_SIZE)
    image_rgb = cv2.bilateralFilter(image_rgb, d=5, sigmaColor=50, sigmaSpace=5
    image_rgb = _apply_clahe(image_rgb)
    return image_rgb

def _resize_with_padding(image: np.ndarray, size: int) -> np.ndarray:
    """Resize keeping aspect ratio, then pad to a square (size x size)."""
    h, w = image.shape[:2]
    scale = size / max(h, w)
    new_h, new_w = int(round(h * scale)), int(round(w * scale))
    resized = cv2.resize(image, (new_w, new_h), interpolation=cv2.INTER_AREA)
    top = (size - new_h) // 2
    bottom = size - new_h - top
    left = (size - new_w) // 2
    right = size - new_w - left
    return cv2.copyMakeBorder(
        resized, top, bottom, left, right,
        borderType=cv2.BORDER_CONSTANT, value=(0, 0, 0),
    )

def _apply_clahe(image_rgb: np.ndarray) -> np.ndarray:
    """Apply CLAHE to the L channel in LAB space for local contrast."""
    lab = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2LAB)
    l_channel, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    l_eq = clahe.apply(l_channel)
```

```
    lab_eq = cv2.merge((l_eq, a, b))
    return cv2.cvtColor(lab_eq, cv2.COLOR_LAB2RGB)
```

Teste de contrato (`tests/test_preprocessing.py`): garante shape `(224, 224, 3)`, dtype `uint8` e determinismo (`np.array_equal(preprocess(x), preprocess(x))`). Esse teste é a evidência verificável de "zero skew" que você cita em entrevista.

---

6. Fase 3 — Augmentation e Dataset (`src/dataset.py`)

Regra de ouro: **augmentation existe apenas no treino**. Validação e serving veem exatamente o mesmo pipeline determinístico. A normalização ImageNet (`mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`) é obrigatória porque o backbone foi pré-treinado com ela.

python

```python
import albumentations as A
from albumentations.pytorch import ToTensorV2
import cv2
import torch
from pathlib import Path
from torch.utils.data import Dataset

from src.preprocessing import preprocess

IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)

train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=15, p=0.5),
    A.RandomBrightnessContrast(brightness_limit=0.2, contrast_limit=0.2, p=0.5),
    A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=15,
                         val_shift_limit=10, p=0.3),
    A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    ToTensorV2(),
])

eval_transform = A.Compose([
    A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    ToTensorV2(),
])

CLASS_MAP = {"00-damage": 1, "01-whole": 0}  # damage = positive class

class CarDamageDataset(Dataset):
    """Binary car damage dataset. Applies the shared preprocess() first,
    then split-specific Albumentations transforms."""

    def __init__(self, split_dir: str, transform: A.Compose):
        self.samples = []
        for class_folder, label in CLASS_MAP.items():
            for path in sorted((Path(split_dir) / class_folder).iterdir()):
                self.samples.append((path, label))
        self.transform = transform

    def __len__(self) -> int:
        return len(self.samples)

    def __getitem__(self, idx: int):
        path, label = self.samples[idx]
        image_bgr = cv2.imread(str(path))
```

```
        image_rgb = preprocess(image_bgr)          # <- single source of truth
        tensor = self.transform(image=image_rgb)["image"]
        return tensor, torch.tensor(label, dtype=torch.long)
```

Escolhas de augmentation são conservadoras e justificáveis: flips e rotações leves simulam ângulos de câmera; brilho/contraste simulam condições de luz. Nada agressivo (sem cutout pesado, sem distorções elásticas) que pudesse apagar o próprio dano que queremos detectar.

---

7. Fase 4 — Modelagem e Treino em Duas Fases (`src/model.py`, `src/train.py`)

7.1 Construção do modelo

Transfer learning com dois backbones para comparação honesta: **ResNet50 (IMAGENET1K_V2)** e **EfficientNet-B0**. A cabeça de classificação é substituída por uma camada linear com 2 saídas (logits + `CrossEntropyLoss`, o que mantém as probabilidades via softmax na inferência).

python

```python
import torch.nn as nn
from torchvision import models

def build_model(arch: str = "resnet50", num_classes: int = 2) -> nn.Module:
    """Build a transfer-learning model with a fresh classification head."""
    if arch == "resnet50":
        model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
        model.fc = nn.Linear(model.fc.in_features, num_classes)
    elif arch == "efficientnet_b0":
        model = models.efficientnet_b0(
            weights=models.EfficientNet_B0_Weights.IMAGENET1K_V1)
        model.classifier[1] = nn.Linear(
            model.classifier[1].in_features, num_classes)
    else:
        raise ValueError(f"Unknown architecture: {arch}")
    return model
```

7.2 Por que duas fases de treino (resposta pronta de entrevista)

Fase 1 — cabeça com backbone congelado: a nova camada linear tem pesos aleatórios; se treinássemos tudo de uma vez, gradientes grandes e ruidosos dela destruiriam as features ImageNet do backbone (catastrophic forgetting). Congelamos o backbone (`requires_grad=False`) e treinamos só a cabeça com lr ~1e-3.

Fase 2 — fine-tuning do último bloco: com a cabeça estabilizada, descongelamos apenas o último bloco convolucional (`layer4` no ResNet50; `features[-1]` no EfficientNet) com lr reduzido (~1e-4/1e-5). Features iniciais (bordas, texturas) são universais; as finais é que

precisam se especializar em "aparência de dano". Descongelar tudo com um dataset deste porte convidaria overfitting.

7.3 Loop de treino — mecanismos de controle

- **Otimizador:** Adam (ou AdamW), lr por fase como acima.
- **Scheduler:** `ReduceLROnPlateau` monitorando `val_loss` (factor=0.5, patience=2) — reduz lr quando a validação estagna.
- **Early stopping:** paciência de ~5 épocas sem melhora em `val_loss`; evita treinar além do ponto de generalização.
- **Checkpoint:** salva `state_dict` apenas quando `val_loss` melhora (`models/best_{arch}.pt`), junto com um JSON de metadados (época, métricas, seed, versão do código).
- **Histórico:** loss/accuracy por época salvos em `outputs/history_{arch}.json` e plotados (curvas de treino vs. validação) para diagnóstico visual de overfitting.

python

```python
# Esqueleto do controle de treino (src/train.py)
best_val_loss = float("inf")
patience, patience_counter = 5, 0

for epoch in range(max_epochs):
    train_loss = train_one_epoch(model, train_loader, optimizer, criterion, dev
    val_loss, val_acc = evaluate(model, val_loader, criterion, device)
    scheduler.step(val_loss)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), f"models/best_{arch}.pt")
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f"Early stopping at epoch {epoch}.")
            break
```

7.4 Reprodutibilidade (evidência de engenharia)

python

```python
import random, numpy as np, torch

def set_seed(seed: int = 42) -> None:
    """Fix all random sources for reproducible experiments."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Somado a `requirements.txt` com **versões pinadas** (`torch==2.x.y`, `opencv-python==4.x.y`, `albumentations==1.x.y`, etc.), qualquer pessoa reproduz o experimento com um `pip install -r requirements.txt && python -m src.train`.

---

8. Fase 5 — Avaliação Orientada a Negócio (`src/evaluate.py`)

A avaliação vai além de accuracy porque o custo dos erros é assimétrico:

1. **Matriz de confusão** sobre o conjunto de validação, salva em `outputs/confusion_matrix_{arch}.png`.
2. **Precision, recall e F1 por classe** (`sklearn.metrics.classification_report`).
3. **ROC-AUC para a classe dano** (`roc_auc_score` sobre as probabilidades softmax da classe positiva).
4. **Análise explícita de falsos negativos:** listar as imagens de dano classificadas como íntegras, inspecioná-las visualmente (danos sutis? fotos escuras? ângulos ruins?) e cruzar com os overlays de Grad-CAM.
5. **Ajuste de threshold:** em vez do argmax padrão (threshold 0.5), varrer thresholds sobre `P(dano)` e escolher um ponto operacional que priorize recall da classe dano, documentando quanto de precision se sacrifica. Exemplo de narrativa defensável: *"reduzindo o threshold de 0.50 para [X], o recall de dano subiu de [Y] para [Z] ao custo de [W] pontos de precision — trade-off aceitável porque o falso negativo encerra indevidamente um sinistro legítimo, enquanto o falso positivo apenas gera uma revisão humana extra." (valores: [MÉTRICAS — preencher])*

```python
```

```python
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, precision_recall_curve)

def evaluate_with_threshold(y_true, probs_damage, threshold: float = 0.5):
    """Compute business-oriented metrics for a given decision threshold."""
    y_pred = (probs_damage >= threshold).astype(int)
    report = classification_report(y_true, y_pred,
                                   target_names=["whole", "damage"],
                                   output_dict=True)
    auc = roc_auc_score(y_true, probs_damage)
    cm = confusion_matrix(y_true, y_pred)
    return {"report": report, "roc_auc": auc, "confusion_matrix": cm}
```

Resultado esperado como referência: val_accuracy na faixa 88–95% *(usar somente o número real após o treino; até lá, [MÉTRICA — preencher])*.

---

9. Fase 6 — Explicabilidade com Grad-CAM (`src/explain.py`)

O que Grad-CAM é neste projeto: um mapa de calor derivado dos gradientes da classe predita em relação às ativações da última camada convolucional (`model.layer4[-1]` no ResNet50). Ele mostra *quais regiões mais influenciaram a decisão*.

O que Grad-CAM não é: segmentação. Não há máscara pixel a pixel, não há IoU/Dice, não há garantia de contorno preciso do dano. O termo correto é **localização fraca (weak localization)** — e é assim que o projeto se apresenta em README, model card e entrevistas.

python

```python
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
from pytorch_grad_cam.utils.model_targets import ClassifierOutputTarget

def generate_gradcam_overlay(model, input_tensor, rgb_float_image,
                             target_class: int, target_layer):
    """Generate a Grad-CAM heatmap overlay for a given class prediction."""
    cam = GradCAM(model=model, target_layers=[target_layer])
    grayscale_cam = cam(input_tensor=input_tensor,
                        targets=[ClassifierOutputTarget(target_class)])[0]
    return show_cam_on_image(rgb_float_image, grayscale_cam, use_rgb=True)
```

Protocolo de análise (salvo em `outputs/gradcam/`):

- Overlays para **acertos e erros das duas classes** (verdadeiros positivos, falsos negativos, verdadeiros negativos, falsos positivos).
- **Verificação de shortcut learning:** se os mapas de calor destacam fundo, placa, chão ou marca d'água em vez da região do dano, o modelo aprendeu atalhos espúrios do

dataset — isso vira uma limitação documentada e motiva augmentation adicional ou curadoria de dados.

---

10. Fase 7 — Serving com FastAPI (`app/main.py`, `app/schemas.py`)

A API importa `preprocess()` de `src/preprocessing.py` — o mesmo módulo do treino. Este é o argumento concreto de consistência treino/serving.

Contrato da API:

Endpoint	Método	Entrada	Saída	Erros
`/health`	GET	—	`{"status": "ok", "model": "<arch>"}`	—
`/predict`	POST	multipart `file` (image/jpeg ou image/png)	`{"prediction", "confidence", "probabilities"}`	415 (content-type inválido), 422 (arquivo ilegível), 500

python

```python
import io
import cv2
import numpy as np
import torch
from fastapi import FastAPI, File, HTTPException, UploadFile

from app.schemas import PredictionResponse
from src.preprocessing import preprocess           # same function as training
from src.dataset import eval_transform
from src.model import build_model

app = FastAPI(title="ClaimSight API", version="1.0.0")

ALLOWED_TYPES = {"image/jpeg", "image/png"}
CLASSES = {0: "whole", 1: "damage"}

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = build_model("resnet50")
model.load_state_dict(torch.load("models/best_resnet50.pt",
                                 map_location=device))
model.to(device).eval()

@app.get("/health")
def health():
    return {"status": "ok", "model": "resnet50"}

@app.post("/predict", response_model=PredictionResponse)
async def predict(file: UploadFile = File(...)):
    if file.content_type not in ALLOWED_TYPES:
        raise HTTPException(status_code=415,
                            detail=f"Unsupported content type: {file.content_ty
    raw = await file.read()
    buffer = np.frombuffer(raw, dtype=np.uint8)
    image_bgr = cv2.imdecode(buffer, cv2.IMREAD_COLOR)
    if image_bgr is None:
        raise HTTPException(status_code=422, detail="Could not decode image.")

    image_rgb = preprocess(image_bgr)
    tensor = eval_transform(image=image_rgb)["image"].unsqueeze(0).to(device)

    with torch.no_grad():
        probs = torch.softmax(model(tensor), dim=1).squeeze().tolist()

    pred_idx = int(np.argmax(probs))
    return PredictionResponse(
        prediction=CLASSES[pred_idx],
```

```
        confidence=round(probs[pred_idx], 4),
        probabilities={"whole": round(probs[0], 4),
                       "damage": round(probs[1], 4)},
    )
```

python

```python
# app/schemas.py
from pydantic import BaseModel

class PredictionResponse(BaseModel):
    prediction: str
    confidence: float
    probabilities: dict[str, float]
```

Execução: `uvicorn app.main:app --host 0.0.0.0 --port 8000`. Testes de contrato em `tests/test_api.py` com `fastapi.testclient.TestClient` cobrindo: health check, predição válida, content-type errado (espera 415) e bytes corrompidos (espera 422).

---

11. Fase 8 — Publicação no Hugging Face Hub (`scripts/publish_hf.py`)

Publicação **versionada** do melhor checkpoint com model card completo:

python

```python
from huggingface_hub import HfApi

api = HfApi()
api.upload_file(
    path_or_fileobj="models/best_resnet50.pt",
    path_in_repo="best_resnet50.pt",
    repo_id="<user>/claimsight-damage-detection",
    commit_message="v1.0 — ResNet50 two-phase fine-tuning, val_loss checkpoint"
)
```

Consumo no README e na API (elimina a necessidade de commitar binários no Git):

python

```python
from huggingface_hub import hf_hub_download

checkpoint_path = hf_hub_download(
    repo_id="<user>/claimsight-damage-detection",
    filename="best_resnet50.pt",
)
```

Model card (seções obrigatórias):

- **Uso pretendido:** triagem de fotos de sinistros com revisão humana; não é decisão final de sinistro.
- **Limitações:** labels apenas binários; Grad-CAM é localização fraca, não segmentação; desempenho não validado em fotos noturnas, veículos parcialmente visíveis ou tipos de dano ausentes do dataset.
- **Viés de dataset:** distribuição de marcas/modelos/cenários do Kaggle pode não representar a carteira real de uma seguradora; recomenda-se validação em dados do domínio antes de qualquer uso.
- **Métricas:** apenas as reais do experimento *([MÉTRICAS — preencher])*.

---

12. Metodologia de Trabalho e Ordem de Execução

O projeto segue um fluxo incremental onde cada fase produz um artefato verificável antes da próxima começar:

1. **Setup e ingestão** → `data/raw/` validado + `dataset_summary()` documentado.
2. **Pré-processamento** → `preprocess()` + testes de determinismo passando + notebook com antes/depois visual de cada etapa (ótimo material de README).
3. **Dataset e DataLoaders** → batch visualizado (sanity check: labels corretos, imagens sem distorção).
4. **Baseline (Fase 1 de treino, só a cabeça)** → primeira métrica real registrada.
5. **Fine-tuning (Fase 2)** → comparação ResNet50 vs. EfficientNet-B0 em tabela.
6. **Avaliação + threshold** → matriz de confusão, ROC-AUC, análise de FN documentada.
7. **Grad-CAM** → overlays das quatro células da matriz de confusão + verificação de shortcut learning.
8. **API** → testes de contrato passando + exemplo de `curl` no README.
9. **Publicação HF Hub + model card** → snippet `hf_hub_download` funcionando de ponta a ponta.
10. **README final** → problema → dados → pipeline → modelagem → resultados → explicabilidade → API → limitações → extensões.

---

13. Limitações Explícitas e Extensões Planejadas

Limitações (documentadas, não escondidas):

- Classificação binária apenas — não distingue tipo (risco, amassado, vidro) nem severidade do dano.
- Grad-CAM fornece localização fraca; não há métricas de localização (IoU/Dice) porque não há ground truth espacial.
- Dataset de escopo limitado; generalização para fotos reais de seguradoras não é garantida.
- Ferramenta de triagem: exige revisão humana em todo o fluxo.

Roadmap de segmentação real (extensão, não capacidade atual):

- **Opção A:** anotar manualmente um subconjunto (~200–500 imagens) com CVAT ou Label Studio e treinar um U-Net via `segmentation-models-pytorch`, reportando IoU/Dice.
- **Opção B:** migrar para o dataset **CarDD**, que possui máscaras de segmentação por tipo de dano.
- **Outras extensões:** classificação multi-classe de tipo de dano, estimativa de severidade, containerização com Docker, monitoramento de drift em produção.

---

14. Stack Consolidada (habilidade → onde aparece)

Habilidade defensável	Evidência no repositório
Deep learning / transfer learning (PyTorch)	`src/model.py`, `src/train.py` (duas fases, scheduler, early stopping)
Visão computacional (OpenCV)	`src/preprocessing.py` (letterbox, bilateral, CLAHE)
Data augmentation (Albumentations)	`src/dataset.py` (transforms separados por split)
Explicabilidade (Grad-CAM)	`src/explain.py`, `outputs/gradcam/`
Avaliação orientada a negócio	`src/evaluate.py` (threshold tuning, análise de FN)
Engenharia de pipeline / zero skew	`preprocess()` única + `tests/test_preprocessing.py`
Reprodutibilidade	`set_seed()`, `requirements.txt` pinado, checkpoints com metadados
APIs de ML (FastAPI)	`app/main.py`, `app/schemas.py`, `tests/test_api.py`
MLOps / publicação de modelos (HF Hub)	`scripts/publish_hf.py`, model card
Fundamentos de segmentação (roadmap)	Seção de extensões (CVAT/Label Studio + U-Net, CarDD)

Documento de escopo do ClaimSight — todas as métricas marcadas como [MÉTRICA — preencher] devem ser substituídas exclusivamente por resultados reais de experimento antes de qualquer publicação.